

Greedy Algorithms

SMITA SANKHE

Assistant Professor

Department of Computer Engineering

Problem Characteristics

- Decomposable?
- Unique solution or set of solutions?
- Any solution or best solution?

Greedy Algorithms

- Typically apply to optimization problems in which we make a set of choices in order to arrive at an optimal solution.
- Used for solving optimization problems.
- The decision is taken based on the currently available information.
- This technique is basically used to determine the feasible solution that may or may not be optimal.
- The feasible solution is a subset that satisfies the given criteria.
- The optimal solution is the solution that is the best and the most favorable solution in the subset.

The general method

```
Greedy(int a[], int n)
{
/* the array a[1]...a[n] contains the input */
1.  solution= $\phi$  /* initialise the solution */
2.  for(i=1;i<=n;i++)
3.  {
4.      j=select(a);
5.      if(feasible(solution,j))
6.  solution=union(solution,j);/* extend the
   solution */
7.  }
8.  return solution;
}/* End of Greedy(...) */
```

Greedy algorithms: The working

- Make a choice at each step.
- Make the choice before solving the subproblems.
- Solve top-down.

Problems typically solved with Greedy Strategy

- Knapsack problem
- Minimum cost spanning trees-Kruskal and prim's algorithm
- Single source shortest path
- Job sequencing with deadlines

Knapsack problem

- Given positive integers $P_1, P_2, \dots, P_n$, $W_1, W_2, \dots, W_n$ and M .
- Find $X_1, X_2, \dots, X_n$, $0 \leq X_i \leq 1$ such that

$$\sum_{i=1}^n P_i X_i \text{ is maximized.}$$

- Subject to
$$\sum_{i=1}^n W_i X_i \leq M$$

Knapsack Problem Example

- $M = 20, (P_1, P_2, P_3) = (25, 24, 15)$
 $(W_1, W_2, W_3) = (18, 15, 10)$
- Three feasible solutions, 1 is optimal

	(X_1, X_2, X_3)	$\sum W_i X_i$	$\sum P_i X_i$
1.	$(1, 2/15, 0)$	20	28.2
2.	$(0, 2/3, 1)$	20	31
3.	$(0, 1, 1/2)$	20	31.5

Knapsack Problem

Example

Find the solution of the knapsack problem for $n=4$, $M=120$
 $(p_1, p_2, p_3, p_4) = (40, 20, 35, 50)$ and $(w_1, w_2, w_3, w_4) = (25, 30, 40, 45)$

Item	1	2	3	4
w	25	30	40	45
p	40	20	35	50
p/w	1.6	0.66	0.875	1.11
x	1	1/3	1	1

Fractional Knapsack Algorithm

Algorithm 1 FractionalKnapsack(S, W)

Input: Set $S = \{i_1, i_2, i_3, \dots, i_n\}$ of items. Each item is assigned with the value v_i and weight w_i . The maximum weight a knapsack can handle is W .

Output: Amount x_i of each item i to maximize benefit with weight at most W .

Repeat for each item $i = 1$ to n

 set $x_i \leftarrow 0$ (selection part of item i is set to zero for every item initially)

 set $p_i \leftarrow v_i / w_i$ {profit value of each item}

End repeat

 set $cw \leftarrow 0$ and $i \leftarrow 1$ {set current total weight to 0 and start index to 1}

Repeat while $cw < W$ and $i \leq n$

 remove item i with highest profit value p_i from set S

 if $(cw + w_i) \leq W$ then

 set $x_i \leftarrow 1$ and $cw \leftarrow cw + w_i$

 else

 set $x_i \leftarrow \{W - cw\} / w_i$

$cw = W$

 endif

 set $i \leftarrow i + 1$

endwhile

return(x)

Fractional Knapsack Algorithm

- **Time Complexity:**

Time complexity of the sorting + Time complexity of the loop to maximize profit

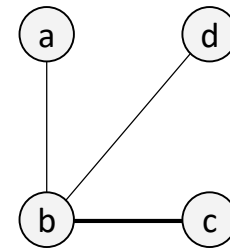
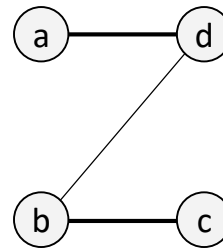
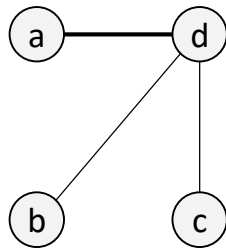
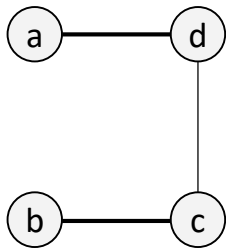
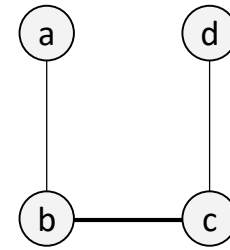
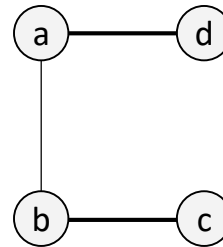
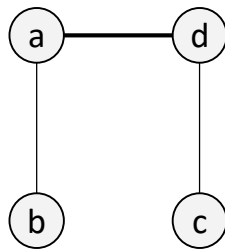
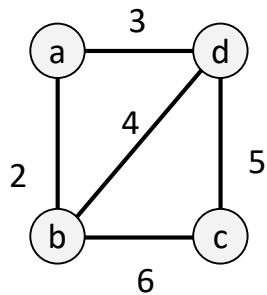
$$= O(N \log N) + O(N) = O(N \log N)$$

- **Space Complexity:** $O(1)$

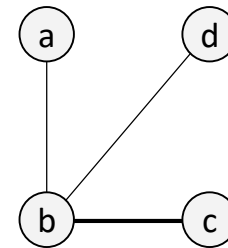
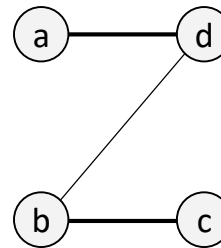
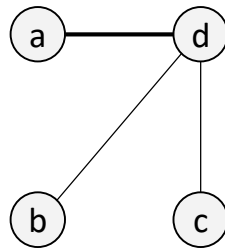
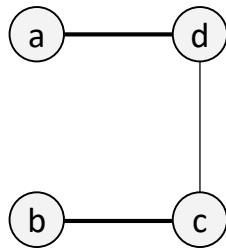
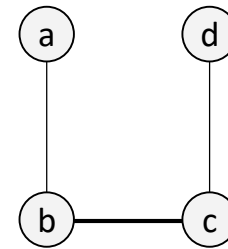
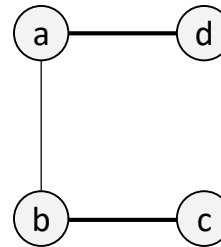
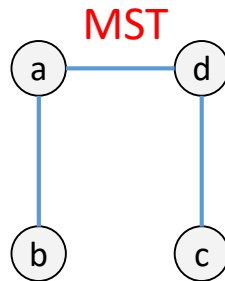
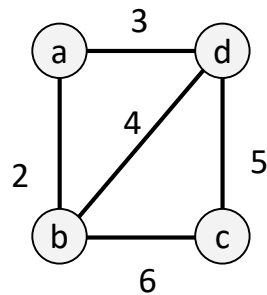
Minimum Spanning tree

- A spanning tree is defined as a subgraph of an undirected graph in which all the vertices are connected.
- A tree consisting of all the vertices of a graph is called a spanning tree.
- The weight of spanning tree for a weighted graph is the sum of all the edge weights.
- The spanning tree with minimum total weight is called the minimum spanning tree(MST).
- MST is a minimal graph G' of a graph G , such that $V(G') = V(G)$ and $E(G') \subseteq E(G)$
- If there are N vertices in the graph, then the MST contains $(N-1)$ edges.

Minimum Spanning tree



Minimum Spanning tree



Minimum spanning trees (MST)

- $G = (V, E)$: weighted connected undirected graph
- **Spanning tree** : $S = (V, T)$, $T \subseteq E$, undirected tree
- **Minimum spanning tree(MST)** : a spanning tree with the smallest total weight.

Minimum Spanning tree

The following are the two different algorithms to find out the MST:

1. Prim's Algorithm
2. Kruskal's Algorithm

Prim's algorithm for finding MST

Step 1: $x \in V$, Let $A = \{x\}$, $B = V - \{x\}$.

Step 2: Select $(u, v) \in E$, $u \in A$, $v \in B$ such that (u, v) has the smallest weight between A and B .

Step 3: Put (u, v) in the tree. $A = A \cup \{v\}$, $B = B - \{v\}$

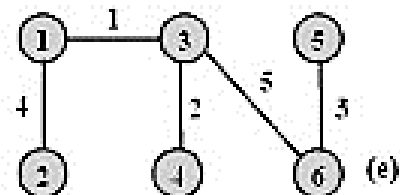
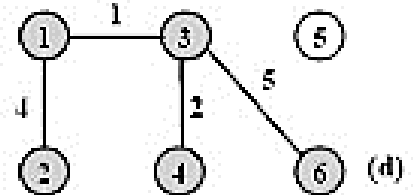
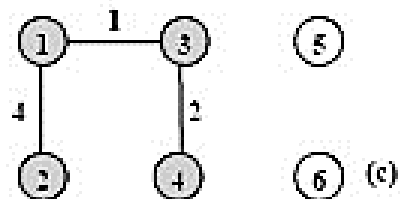
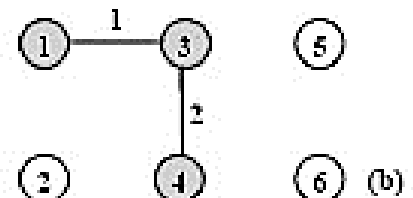
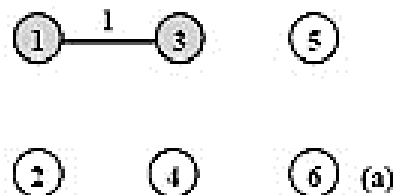
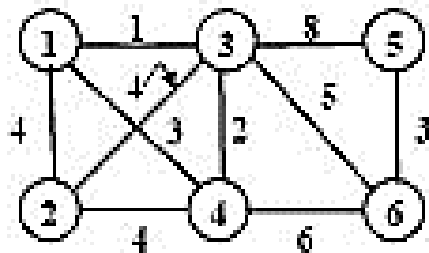
Step 4: If $B = \emptyset$, stop; otherwise, go to Step 2.

- Time complexity : $O(n^2)$, $n = |V|$.

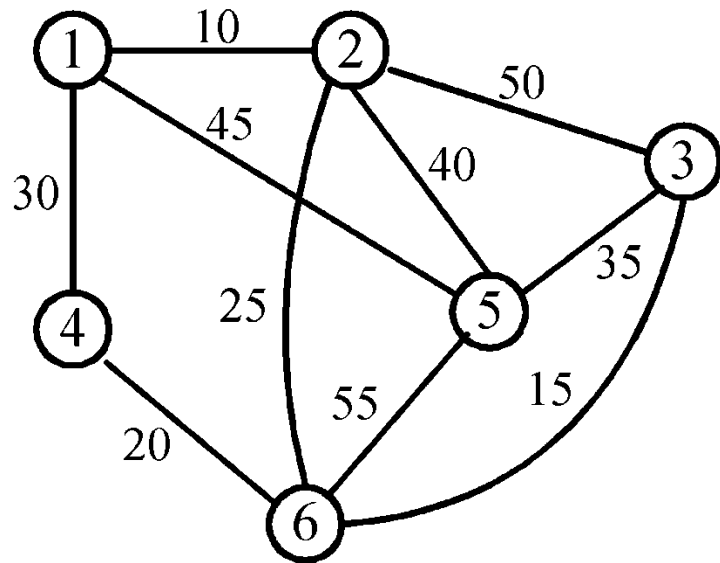
(see the example on the next page)

Prim's algorithm

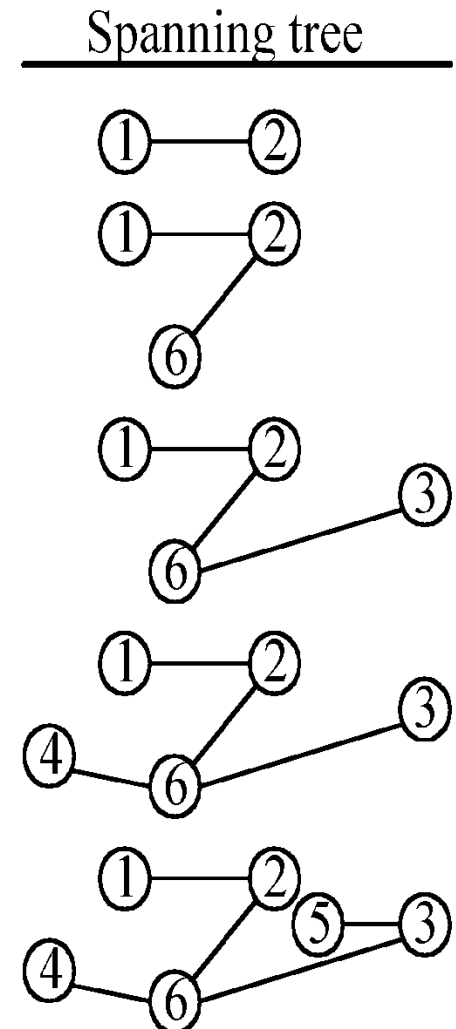
Example 1:



An example for Prim's algorithm

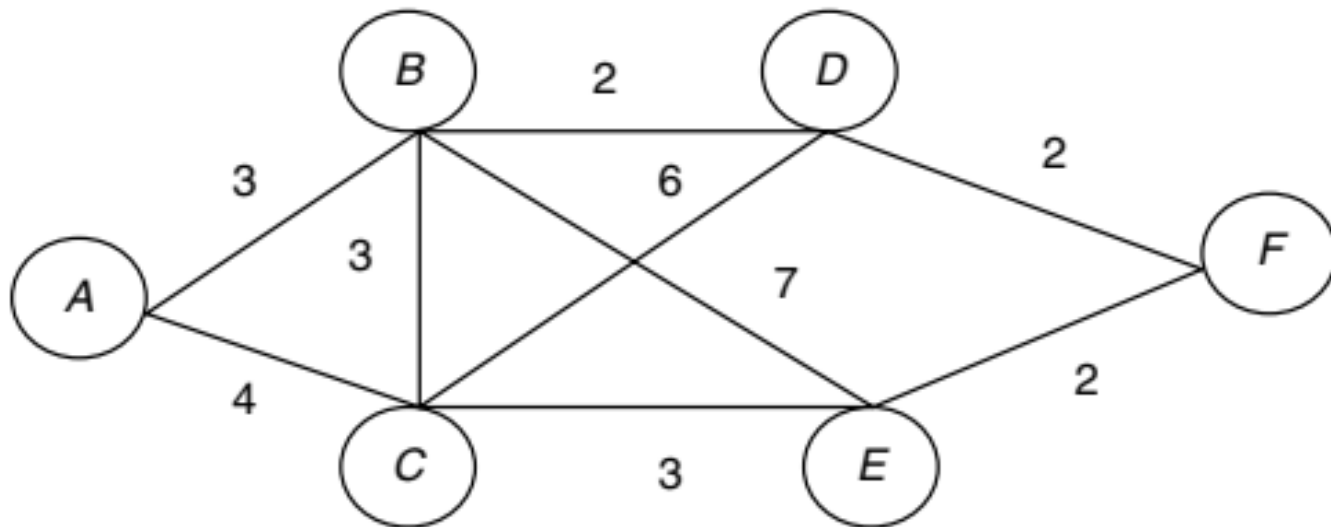


<u>Edge</u>	<u>Cost</u>
(1,2)	10
(2,6)	25
(3,6)	15
(6,4)	20
(3,5)	35



Prim's algorithm

Example 2:



Prim's algorithm

Algorithm Prim(adjMatrix[1 .. n] [1 .. n])

***n** = Number of nodes in the graph*

***nEdges** = Current number of edges included in the MST.*

***min** = Minimum weight (cost) of edge.*

***row, col** indicate the row and column of the edge included in the MST.*

***sum** = Total minimum weight (cost) of the edges included in the MST.*

Initialize selected[] with zeros.

Prim's algorithm

Algorithm Prim(adjMatrix[1 .. n] [1 .. n])

n = Number of nodes in the graph

nEdges = Current number of edges included in the MST.

min = Minimum weight (cost) of edge.

row, col indicate the row and column of the edge included in the MST.

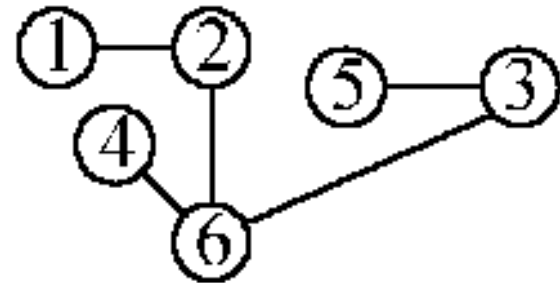
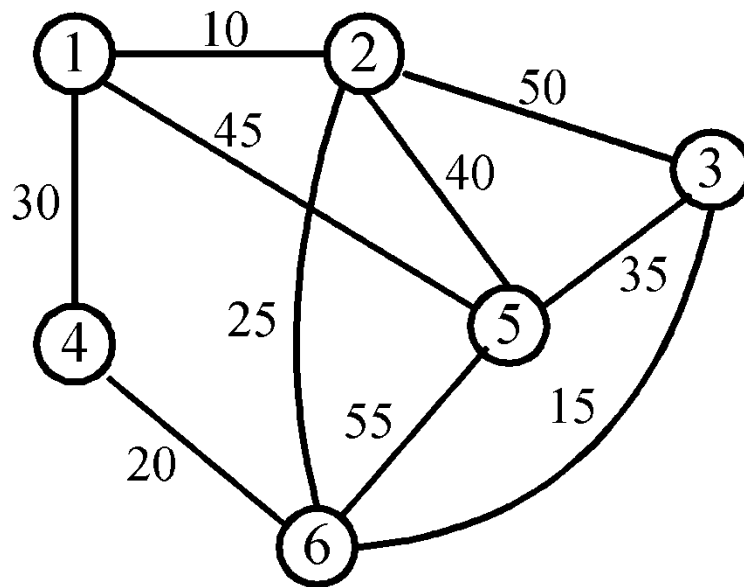
sum = Total minimum weight (cost) of the edges included in the MST.

Initialize selected[] with zeros.

```
selected[1] = 1; // To start with, node 1 is included in MST
nEdges = 1;      // Make one count of edges to be included in MST
sum = 0;
While( nEdges < n)
min = ∞;
/* Repeat following steps to cover all nodes from 1 to n */
For i = 1 to n do
If( selected[i] = 1)
/* If node i is included in MST, then repeat to get min-cost node j to
connect to i */
for j = 1 to n do
If( selected[i] = 0 ) // Select the min-cost edge
If( min > adjMatrix[i][j] )
min = adjMatrix[i][j];
row = i; col = j;
Endif
Endif
Endfor
Print row, col, adjMatrix[row][col];
Endfor
selected[col] = 1; // Once i and j form a min-edge, mark 1
selected
nEdges++;
sum = sum + adjMatrix[row][col];
Endwhile.
Print sum; // Total Min-Cost
```

An example of MST

- A graph and one of its minimum costs spanning tree



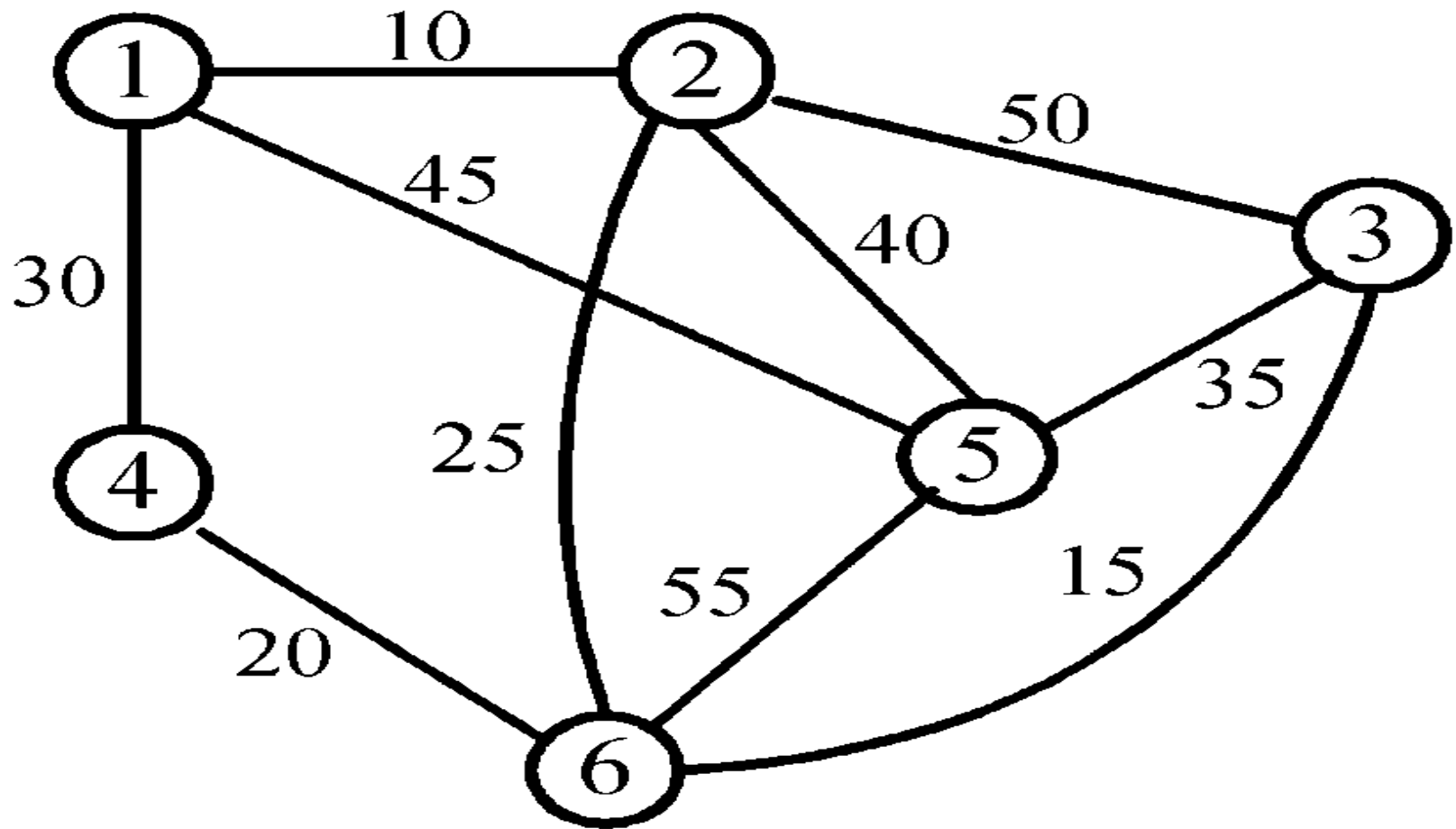
Kruskal's algorithm for finding MST

Step 1: Sort all edges into nondecreasing order.

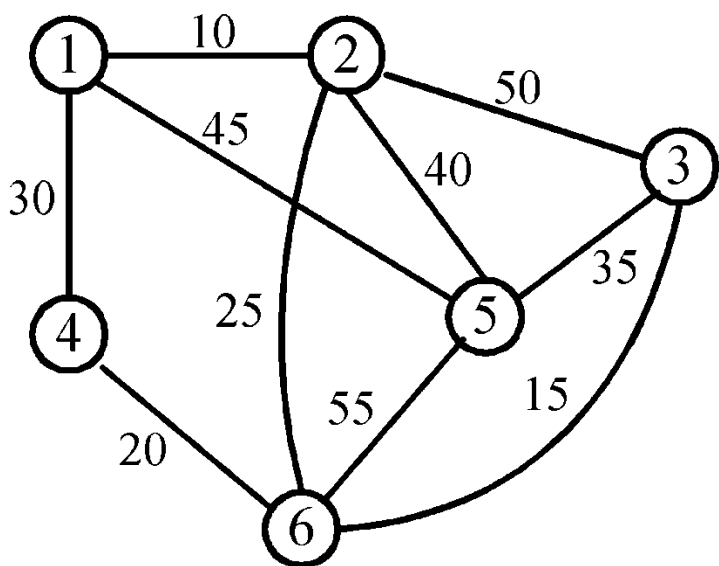
Step 2: Add the next smallest weight edge to the forest if it will not cause a cycle.

Step 3: Stop if $n-1$ edges. Otherwise, go to Step2.

An example of Kruskal's algorithm



An example of Kruskal's algorithm



Edge

Cost

Spanning Forest

(1,2)

10

① ② ③ ④ ⑤ ⑥

(3,6)

15

①—② ③ ④ ⑤ ⑥

(4,6)

20

①—② ③ ④ ⑤
⑥

(2,6)

25

①—② ③ ⑤
④—⑥

(1,4)

30

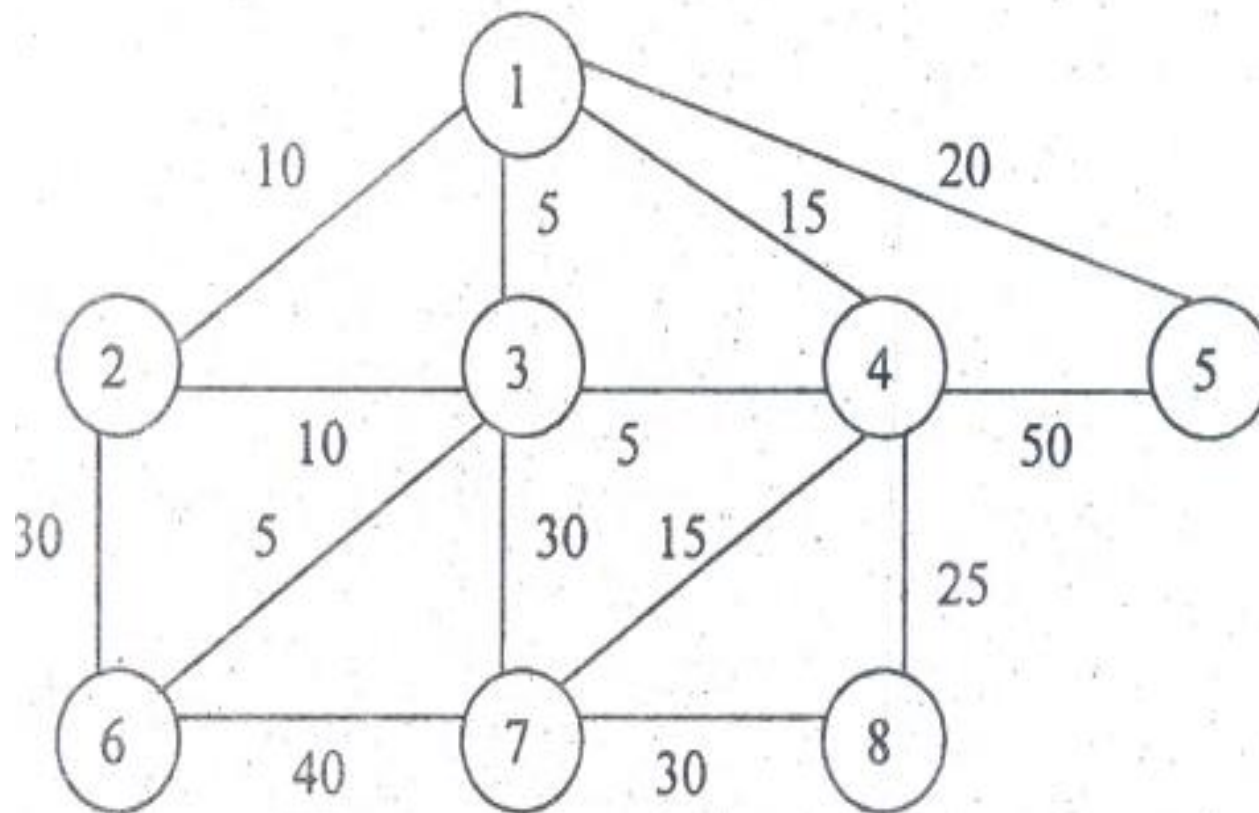
①—② ⑤
④—⑥—③

(3,5)

35

(reject)

①—② ⑤—③
④—⑥



The details for constructing MST

- How do we check if a cycle is formed when a new edge is added?
 - By the SET and UNION method.
- A tree in the forest is used to represent a SET.
- If $(u, v) \in E$ and u, v are in the same set, then the addition of (u, v) will form a cycle.
- If $(u, v) \in E$ and $u \in S_1, v \in S_2$, then perform UNION of S_1 and S_2 .

- Connected undirected weighted graph without self-loops and multiple edges is given. Graph contains n vertices and m edges.
- For each edge (u, v) find the minimal possible weight of the spanning tree that contains the edge (u, v) .
- The weight of the spanning tree is the sum of weights of all edges included in spanning tree.

Input

- First line contains two integers n and m ($1 \leq n \leq 2 \cdot 10^5$, $n - 1 \leq m \leq 2 \cdot 10^5$) — the number of vertices and edges in graph.
- Each of the next m lines contains three integers u_i, v_i, w_i ($1 \leq u_i, v_i \leq n$, $u_i \neq v_i$, $1 \leq w_i \leq 10^9$) — the endpoints of the i -th edge and its weight.

Output

- Print m lines. i -th line should contain the minimal possible weight of the spanning tree that contains i -th edge.
- The edges are numbered from 1 to m in order of their appearing in input.

Applications of - MST

- **Network design.**
 - *telephone, electrical, hydraulic, TV cable, computer, road*
- **Approximation algorithms for NP-hard problems.**
 - [*traveling salesperson problem*](#), [*Steiner tree*](#)
- **Indirect applications.**
 - max bottleneck paths
 - codes for error correction
 - learning salient features for real-time face verification
 - reducing data storage in sequencing amino acids in a protein
 - model locality of particle interactions in turbulent fluid flows
 - autoconfig protocol for Ethernet bridging to avoid cycles in a network
- [**Circuit design**](#): implementing efficient multiple constant multiplications, as used in [**finite impulse response**](#) filters.
- [**Regionalisation**](#) of socio-geographic areas, the grouping of areas into homogeneous, contiguous regions.

Single Source Shortest Path

- Problem Definition

Given a directed graph $G = \langle V, E \rangle$, a weighting function cost for edges of G and a source vertex v_0 , the problem is to compute the shortest paths from v_0 to all the remaining vertices of G .

Assumption:- all weights are positive.

Shortest path between v_0 to some other node v is an ordering among a subset of edges.

Algorithm

```
1  Algorithm ShortestPaths( $v, cost, dist, n$ )
2  //  $dist[j]$ ,  $1 \leq j \leq n$ , is set to the length of the shortest
3  // path from vertex  $v$  to vertex  $j$  in a digraph  $G$  with  $n$ 
4  // vertices.  $dist[v]$  is set to zero.  $G$  is represented by its
5  // cost adjacency matrix  $cost[1 : n, 1 : n]$ .
6  {
7      for  $i := 1$  to  $n$  do
8      { // Initialize  $S$ .
9           $S[i] := \mathbf{false}$ ;  $dist[i] := cost[v, i]$ ;
10     }
11      $S[v] := \mathbf{true}$ ;  $dist[v] := 0.0$ ; // Put  $v$  in  $S$ .
12     for  $num := 2$  to  $n - 1$  do
13     {
14         // Determine  $n - 1$  paths from  $v$ .
15         Choose  $u$  from among those vertices not
16         in  $S$  such that  $dist[u]$  is minimum;
17          $S[u] := \mathbf{true}$ ; // Put  $u$  in  $S$ .
18         for (each  $w$  adjacent to  $u$  with  $S[w] = \mathbf{false}$ ) do
19             // Update distances.
20             if ( $dist[w] > dist[u] + cost[u, w]$ ) then
21                  $dist[w] := dist[u] + cost[u, w]$ ;
22     }
23 }
```

Single Source Shortest Path Conditions

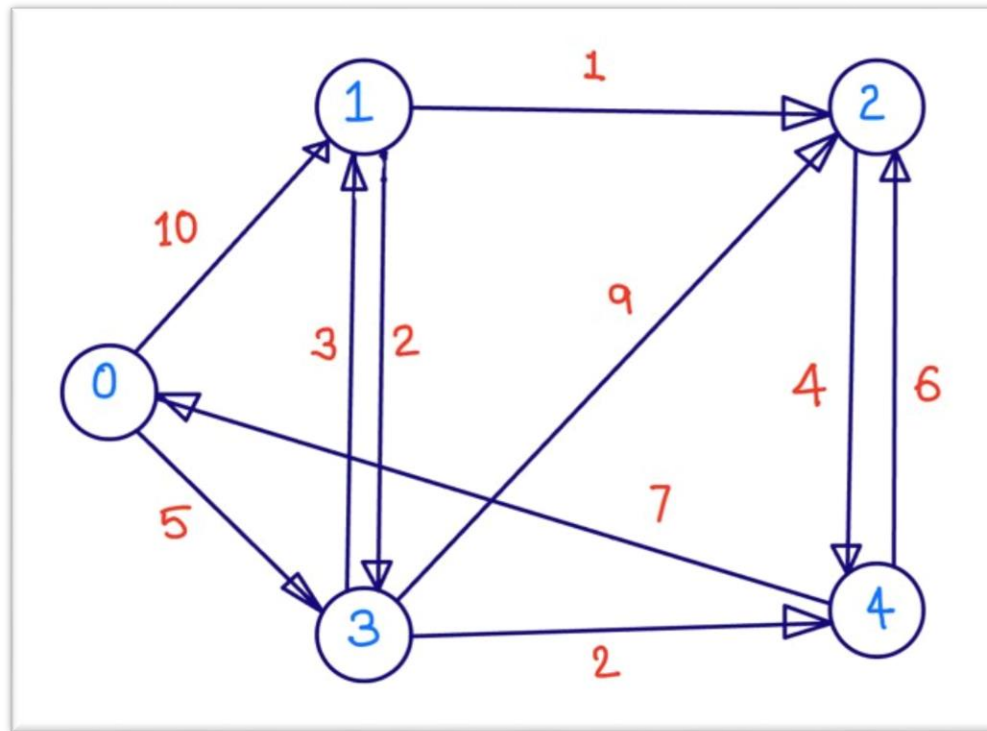
- Feasibility condition:- All possible paths with source vertex i.e a. $n-1$ possible paths and
b. possible iterations $n-2$ excluding source and destination vertex
- Optimal condition :- path that includes all vertices with minimum cost

Dijkstra's Algorithm

Example:

Adjacency Matrix:

	0	1	2	3	4
0	0	10	∞	5	∞
1	∞	0	1	2	∞
2	∞	∞	0	∞	4
3	∞	3	9	0	2
4	7	∞	6	∞	0



Dijkstra's Algorithm

Example:

Relaxation Formula:

If $\text{dist}[z] > \text{dist}[u] + w[u,z]$, then
 $\text{dist}[z] = \text{dist}[u] + w[u,z]$

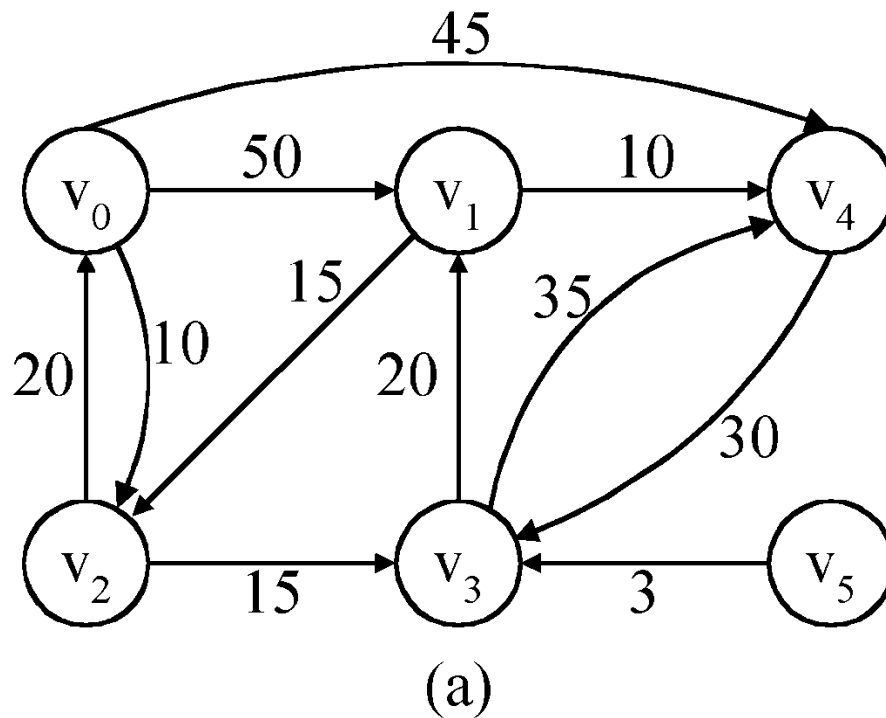
Table 6.4 Shortest path from source vertex 0

Iteration	Set of nodes to which the shortest path is found	Vertex selected	Distance				
			0	1	2	3	4
Initial	{0}	-	0	10	∞	5	∞
1	{0, 3}	3	0	8	14	5	7
2	{0, 3, 4}	4	0	8	13	5	7
3	{0, 3, 4, 1}	1	0	8	9	5	7
4	{0, 3, 4, 1, 2}	2	0	8	9	5	7

The time complexity of the algorithm is $O(n^2)$.

The single-source shortest path problem

- shortest paths from v_0 to all destinations



(b)

	<u>Path</u>	<u>Length</u>
1)	$v_0 v_2$	10
2)	$v_0 v_2 v_3$	25
3)	$v_0 v_2 v_3 v_1$	45
4)	$v_0 v_4$	45

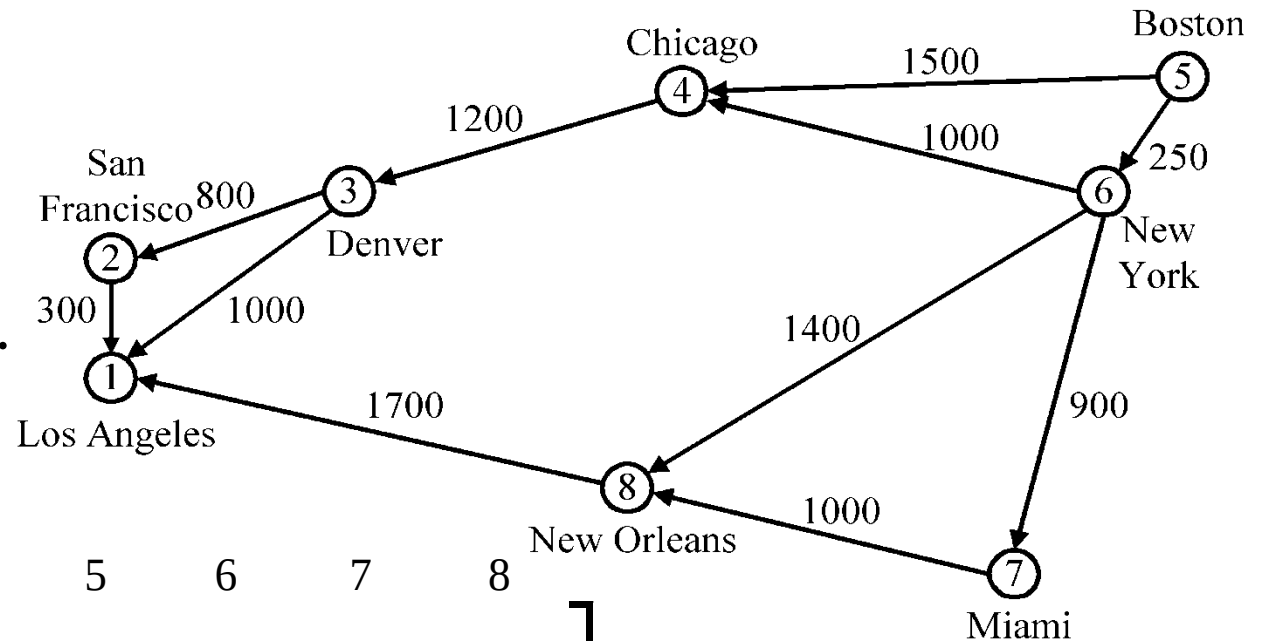
Applications of - SSSP

- automatically find directions between physical locations, such as driving directions on [web mapping](#) websites
- When the problem is represented as a FSA, can be used to find an optimal sequence of choices to reach a certain goal state, or to establish lower bounds on the time needed to reach a given state.
- In a networking or telecommunications mindset, this shortest path problem is sometimes called the min-delay path problem and usually tied with a [widest path problem](#)
- plant and facility layout, [robotics](#), [transportation](#), and [VLSI](#) design

Applications of - SSSP

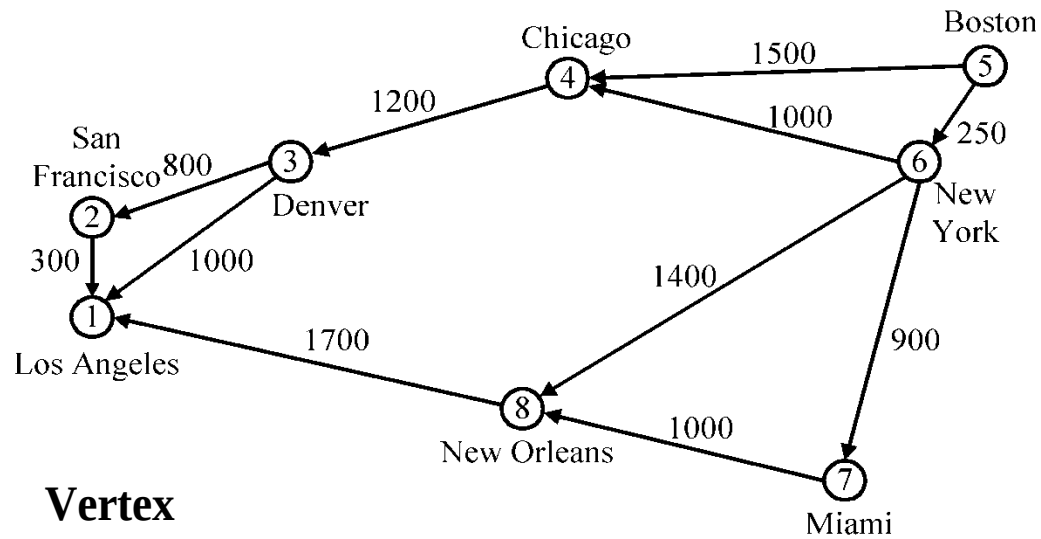
- Maps
- Robot navigation.
- Texture mapping.
- Urban traffic planning.
- Optimal pipelining of VLSI chip.
- Subroutine in advanced algorithms.
- Telemarketer operator scheduling.
- Routing of telecommunications messages.
- Approximating piecewise linear functions.
- Network routing protocols (OSPF, BGP, RIP).
- Exploiting arbitrage opportunities in currency exchange.
- Optimal truck routing through given traffic congestion pattern

Dijkstra's algorithm



Cost adjacency matrix.
All entries not shown
are $+\infty$.

	1	2	3	4	5	6	7	8
1	0							
2	300	0						
3	1000	800	0					
4			1200	0				
5			1500	0	250			
6			1000		0	900	1400	
7						0	1000	
8	1700							0



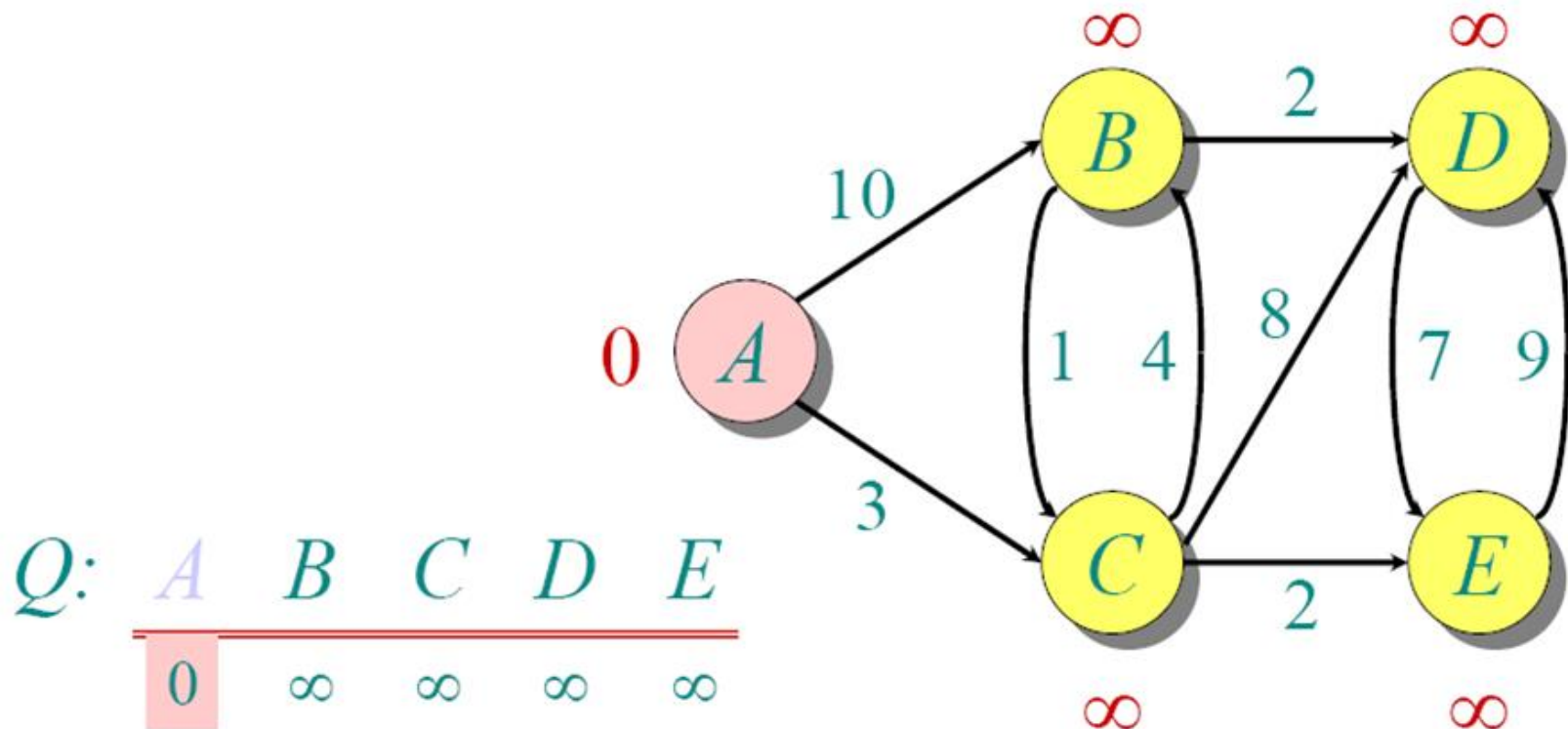
Vertex										
Iteration	S	Selected	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)
Initial		----								
1	5	6	$+\infty$	$+\infty$	$+\infty$	1500	0	250	$+\infty$	$+\infty$
2	5,6	7	$+\infty$	$+\infty$	$+\infty$	1250	0	250	1150	1650
3	5,6,7	4	$+\infty$	$+\infty$	$+\infty$	1250	0	250	1150	1650
4	5,6,7,4	8	$+\infty$	$+\infty$	2450	1250	0	250	1150	1650
5	5,6,7,4,8	3	3350	$+\infty$	2450	1250	0	250	1150	1650
6	5,6,7,4,8,3	2	3350	3250	2450	1250	0	250	1150	1650
	5,6,7,4,8,3,2		3350	3250	2450	1250	0	250	1150	1650

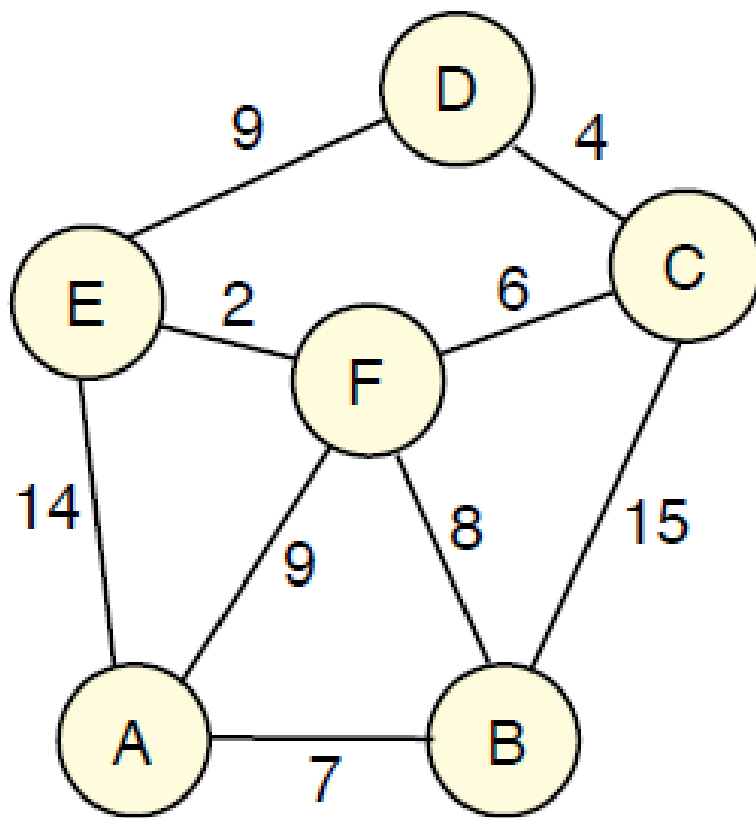
Complexity

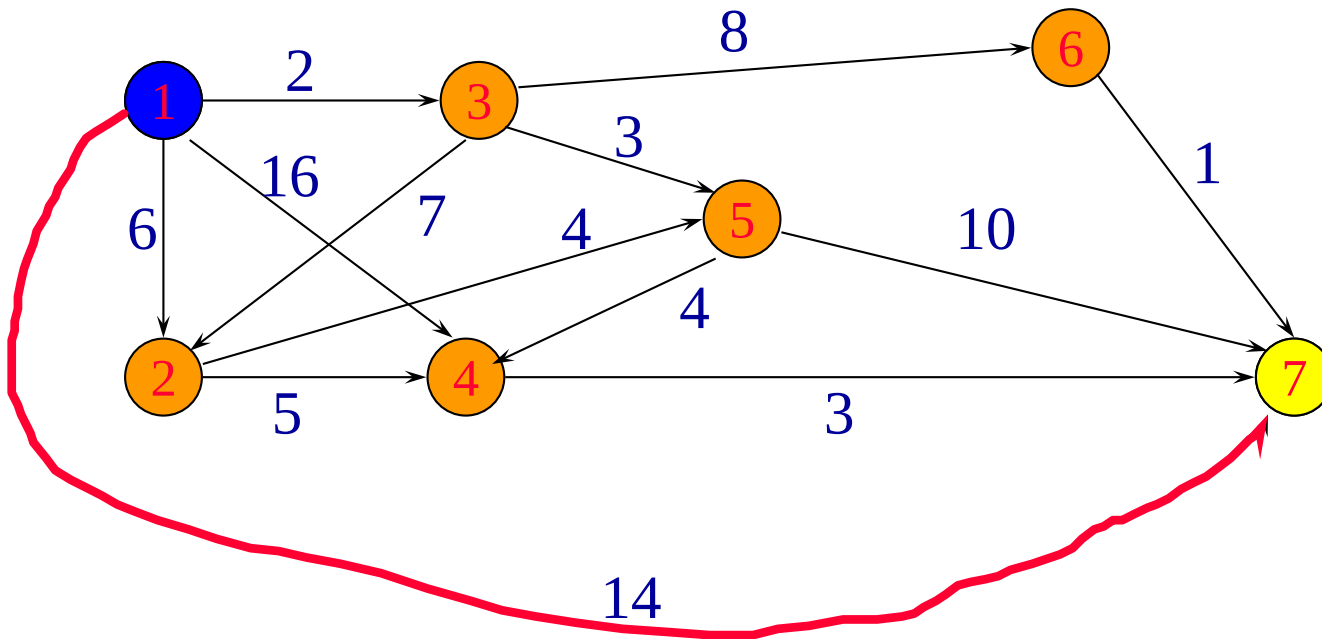
- Overall operations-
= N vertices * (n-2) iterations
= $n(n-2)$
= $n^2 - 2n$
= $O(n^2)$

SSSP Solution

- Problem definition
- Feasibility & Optimality Condition
- OR
- Explain how the answer will be optimal
- Compute
- Final answer
 - Cost of every destination
 - Path to every destination
- complexity







JOB SEQUENCING WITH DEADLINES

Problem Statement

The problem is stated as below.

- There are n jobs to be processed on a machine.
- Each job i has a deadline $d_i \geq 0$ and profit $p_i \geq 0$.
- P_i is earned iff the job is completed by its deadline.
- The job is completed if it is processed on a machine for unit time.
- Only one machine is available for processing jobs.
- Only one job is processed at a time on the machine.

Problem Statement (Contd..)

- A feasible solution is a subset of jobs J such that each job is completed by its deadline.
- An optimal solution is a feasible solution with maximum profit value.

Example : Let $n = 4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$,
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

Example : Let $n = 4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$,
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

Solution

Sr.No.	Feasible Solution	Processing Sequence	Profit value
(i)	(1,2)	(2,1)	110
(ii)	(1,3)	(1,3) or (3,1)	115
(iii)	(1,4)	(4,1)	127 is the optimal one
(iv)	(2,3)	(2,3)	25
(v)	(3,4)	(4,3)	42 
(vi)	(1)	(1)	100
(vii)	(2)	(2)	10
(viii)	(3)	(3)	15
(ix)	(4)	(4)	27

GREEDY ALGORITHM TO OBTAIN AN OPTIMAL SOLUTION

- Consider the jobs in the non increasing order of profits subject to the constraint that the resulting job sequence J is a feasible solution.

```

line  procedure JS( $D, J, n, k$ )
        //  $D(i) \geq 1, 1 \leq i \leq n$  are the deadlines,  $n \geq 1$ . The jobs are//
        // ordered such that  $p_1 \geq p_2 \geq \dots \geq p_n$ .  $J(i)$  is the  $i$ th job in//
        // the optimal solution,  $1 \leq i \leq k$ . Also, at termination  $D(J(i))$ //
        //  $\leq D(J(i + 1)), 1 \leq i < k$ .//
1      integer  $D(0:n), J(0:n), i, k, n, r$ 
2       $D(0) \leftarrow J(0) \leftarrow 0$  //initialize//
3       $k \leftarrow 1; J(1) \leftarrow 1$  //include job 1//
4      for  $i \leftarrow 2$  to  $n$  do //consider jobs in nonincreasing order of  $p_i$ //
        //Find position for  $i$  and check feasibility of insertion//
5         $r \leftarrow k$ 
6        while  $D(J(r)) > D(i)$  and  $D(J(r)) \neq r$  do
7           $r \leftarrow r - 1$ 
8        repeat
9          if  $D(J(r)) \leq D(i)$  and  $D(i) > r$  then
            //insert  $i$  into  $J$ //
10         for  $l \leftarrow k$  to  $r + 1$  by  $-1$  do
11            $J(l + 1) \leftarrow J(l)$ 
12         repeat
13            $J(r + 1) \leftarrow i; k \leftarrow k + 1$ 
14         endif
15       repeat
16     end JS

```

COMPLEXITY ANALYSIS OF JS ALGORITHM

- Let n be the number of jobs and s be the number of jobs included in the solution.
- The loop between lines 4-15 (the for-loop) is iterated $(n-1)$ times.
- Each iteration takes $O(k)$ where k is the number of existing jobs.

$\therefore$ The time needed by the algorithm is $O(sn)$

Since $s \leq n$ so the worst-case time is $O(n^2)$.

If $d_i = n - i + 1 \quad 1 \leq i \leq n$, JS takes $\theta(n^2)$ time

D and J need $\theta(s)$ amount of space.

EXAMPLE:

let $n = 5$, $(p_1, \dots, p_5) = (20, 15, 10, 5, 1)$ and $(d_1, \dots, d_5) = (2, 2, 1, 3, 3)$.

Let $n=7$, $(p_1, \dots, p_7) = (35, 30, 25, 20, 15, 12, 5)$ and $(d_1 \dots d_7) = (3, 4, 4, 2, 3, 1, 2)$

Solutioning methods

- Brute force approach
- Greedy algorithms
 - Greedy strategy 1- Execute the programs with highest profit first to earn more profit (consider jobs in descending order)
 - Greedy strategy 2 - Execute maximum possible programs in their deadlines
(Delay the execution of program maximum possible so that more programs of earlier deadline could be finished to earn more profit.)
 - Greedy Strategy 3 :- (combine both the policies) Execute the programs with highest profit first by delaying their execution maximum possible.